Computer Organization and Architecture (25ESCS04)

Problem Based Learning – II | Units III, IV & V | Maximum Marks: 20

Name: _____ USN: _____ Section: _____

Q1. Computer Arithmetic & Instruction Design (4 Marks)

a) Booth's multiplication: −6 × +5 (2 Marks)

Using five-bit 2's complement: M = −6 = 11010; −M = +6 = 00110; Q = +5 = 00101; initially AC = 00000 and Q−1 = 0. For (Q0,Q−1): 00/11 → no operation; 01 → AC + M; 10 → AC − M. After each operation, arithmetically shift right the combined AC,Q,Q−1 registers.

Cycle	Q0 Q−1	Operation	AC before shift	AC after shift	Q after shift	Q−1
Initial	—	Initialize	00000	00000	00101	0
1	10	AC − M	00110	00011	00010	1
2	01	AC + M	11101	11110	10001	0
3	10	AC − M	00100	00010	01000	1
4	01	AC + M	11100	11110	00100	0
5	00	No operation	11110	11111	00010	0

Final product AC:Q = 11111 00010 = 1111100010 (10-bit 2's complement). Its magnitude is obtained by complementing and adding one: 0000011101 + 1 = 0000011110 = 30. Therefore −6 × 5 = −30.

b) Addressing modes (1 Mark)

I would load the two operands from their fixed memory addresses into registers, then multiply them using register addressing. For example: LOAD R1, [1000]; LOAD R2, [1001]; MUL R1, R2. The multiplication can then use the register values without reading the operands from memory again.

c) RISC or CISC (1 Mark)

I would choose RISC for this calculator. Simple instruction decoding can reduce the control hardware needed, which suits a low-cost design. The load-store approach also separates memory access from arithmetic and makes pipelining easier to organize.

Q2. I/O Organization & Interrupts (4 Marks)

a) Priority-interrupt scheme (2 Marks)

Priority	Sensor	Reason
1 (highest)	Pressure	Unsafe pressure may demand

		immediate protective action.
2	Temperature	Overheating can damage equipment and needs rapid response.
3	Vibration	Fault alerts matter, while continuous bulk logging can use DMA.
4 (lowest)	Flow	Routine monitoring generally tolerates a short delay.

The controller should handle the highest-priority pending interrupt first. With nested interrupts enabled, a more urgent request can interrupt a lower-priority service routine. The order shown above is a proposed order; the final priorities depend on the factory's safety requirements.

b) DMA vs. programmed I/O (1 Mark)

Programmed I/O makes the CPU check the device and copy the data itself. For continuous vibration logs, I would use DMA. The CPU sets up the transfer, and the DMA controller copies the data to memory and signals completion. This gives the CPU more time to handle urgent sensor interrupts.

c) Isolated vs. memory-mapped I/O (1 Mark)

Isolated I/O	Memory-mapped I/O
Separate I/O address space.	Device registers share the memory address space.
Special instructions (such as IN/OUT).	Ordinary LOAD/STORE instructions.
Does not consume memory addresses.	Reserves address ranges for device registers.

I would use memory-mapped I/O for this controller. The CPU can access the sensor registers with the same load and store instructions it uses for memory.

Q3. Pipelining & Vector Processing (4 Marks)

a) Pipeline hazards (2 Marks)

Hazard	Rendering example
Structural	Texture and vertex fetches simultaneously compete for a shared memory port.
Data (RAW)	A shading instruction needs coordinates that a previous instruction has not yet produced.
Control	A conditional pixel-discard branch changes which instruction should execute next.

For example, MUL R1, R2, R3 followed by ADD R4, R1, R5 has a read-after-write (RAW) dependency. The ADD needs the value of R1 produced by the MUL, so it must wait if that result is not yet available.

b) One mitigation for each (1 Mark)

Separate memory paths or additional hardware units can reduce structural hazards. Forwarding can pass a result directly to a dependent instruction, though some data hazards still require a stall. Branch prediction reduces the delay from control hazards when the prediction is correct. If it is wrong, the processor discards the instructions fetched along the wrong path.

c) Vector processing advantage (1 Mark)

Graphics processing applies many of the same operations to different vertices and pixels. A vector instruction such as VADD C, A, B adds corresponding elements of two vectors. It replaces several scalar additions with one instruction, reducing instruction overhead and improving throughput for suitable rendering operations.

Q4. Memory Hierarchy & Cache Design (4 Marks)

a) Conflict misses: direct mapped vs. four-way (2 Marks)

Consider two caches, each with 16 lines. In a direct-mapped cache, line = block mod 16. For the sequence 0, 16, 32, 48, 0, 16, 32, 48, every block maps to line 0. Each access replaces the block already there, so all eight accesses miss. A four-way cache of the same capacity has four sets, with set = block mod 4. These blocks still map to set 0, but its four ways can hold all four blocks. The first four accesses miss; the next four hit. LRU selects a block to replace only when a new block needs space in a full set. This example shows how associativity reduces conflict misses.

b) LRU trace (1 Mark)

Reference string: 3, 1, 4, 1, 5, 3, 2, 1; one four-way set. Contents below are ordered least recently used → most recently used.

Step	Block	Set contents (LRU → MRU)	Result
1	3	3	Miss
2	1	3, 1	Miss
3	4	3, 1, 4	Miss
4	1	3, 4, 1	Hit
5	5	3, 4, 1, 5	Miss
6	3	4, 1, 5, 3	Hit
7	2	1, 5, 3, 2 (evict 4)	Miss
8	1	5, 3, 2, 1	Hit

Totals: 3 hits, 5 misses; hit ratio = 3/8 = 37.5%; miss ratio = 5/8 = 62.5%.

c) Write policy (1 Mark)

I would use write-back for the server to reduce main-memory traffic. A write updates the cached line and marks it dirty. The changed data is written to lower-level memory when the line is evicted. Repeated writes to the same cached line therefore avoid repeated memory writes. Cache coherence and data protection still need attention because main memory may hold an older copy.

Q5. ARM SoC Case Study (4 Marks)

a) RISC/load-store suitability (1 Mark)

In a Cortex-M processor, arithmetic mainly uses registers, while load and store instructions access memory. Simple decoding and compact Thumb encodings help limit hardware and program size. These features suit a fitness tracker, where cost and battery life matter. Thumb instructions may be 16 or 32 bits, depending on the instruction and the core.

b) Pipeline benefit and hazard (2 Marks)

A three-stage pipeline can overlap instruction fetch (IF), decode (ID), and execution (EX). The example below shows three instructions moving through these stages, assuming no stalls:

Cycle	Instruction 1	Instruction 2	Instruction 3
1	IF	—	—
2	ID	IF	—
3	EX	ID	IF
4	—	EX	ID
5	—	—	EX

A load-use hazard can occur with LDR R1, [R0] followed by ADD R2, R1, R3. The ADD depends on the value being loaded into R1 and may reach execution before that value is ready. Forwarding, instruction scheduling, or a stall can handle this, depending on the implementation. The table is a conceptual example; actual pipeline details vary between Cortex-M models.

c) Memory hierarchy (1 Mark)

For the tracker, I would use registers for current operands, on-chip SRAM for sensor buffers and working data, and Flash for firmware. A small cache may help if the selected Cortex-M core supports it and the workload benefits. A large cache could add area and power consumption with little benefit for a simple wearable.

Conclusion

These examples show why hardware choices depend on the workload. A calculator benefits from simple arithmetic hardware, while a factory controller needs timely interrupt handling. Graphics processing benefits from vector operations, and a busy server benefits from fewer memory accesses. For a fitness tracker, the design must also keep power use low.